

Professional Certificate Programme in Agentic AI and Applications



RAG



Revisit the Learning So Far

Let's take a quick look at the key ideas and concepts you have explored in your learning journey.

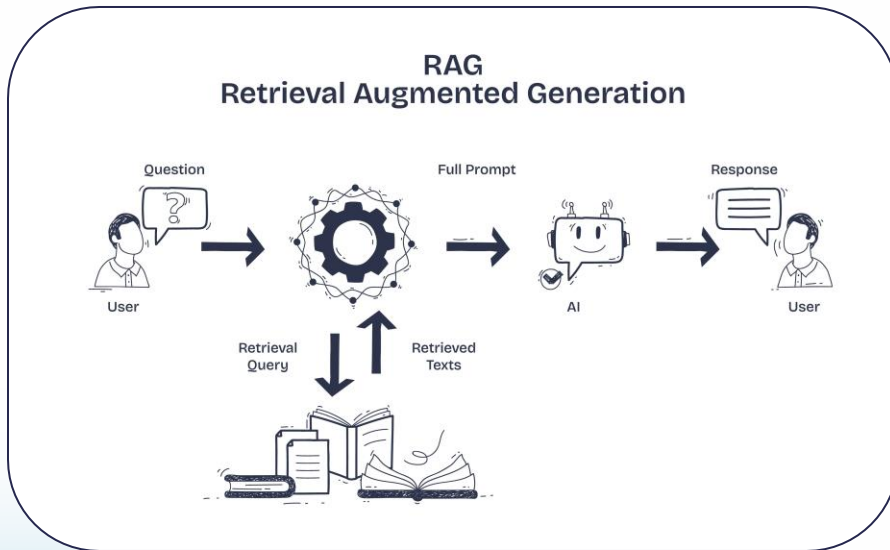
- Learning and adaptation mechanisms used by AI agents to improve performance over time
- Reinforcement learning enables agents to learn optimal actions using rewards and penalties
- Human feedback in RLHF improves alignment, usability and ethical decision-making
- Adaptive behaviour allows agents to adjust continuously, delivering measurable real-world impact

Topics Covered

- What is RAG
- Contextual Embeddings & Relevance Ranking
- Customising RAG Models

What is RAG

What is RAG



- Refers to an AI technique where an LLM retrieves relevant information from external knowledge sources such as PDFs, databases, websites or internal documents
- Uses the retrieved information to generate accurate, grounded and up-to-date answers

RAG = LLM + Search

The system first **retrieves** the right information → then the LLM **generates** the response using that information.

What is RAG

Imagine an AI has two parts to its brain:



"Textbook"
(Its Knowledge Base):
A collection of reliable,
up-to-date information



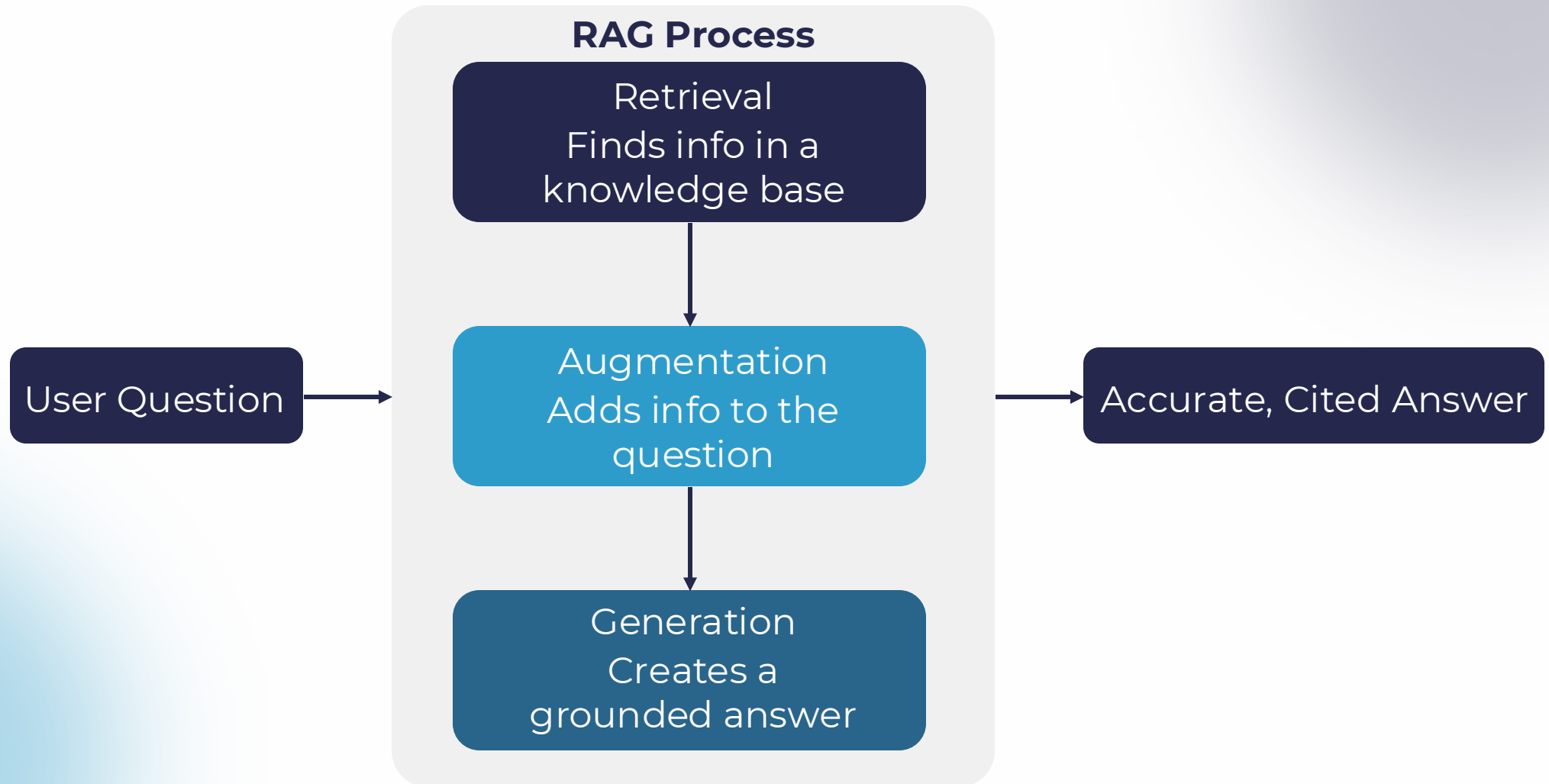
RAG connects both parts



A "Smart Student"
(The Generator):
Great at writing and
explaining answers

- The student first looks up the most relevant facts from the textbook.
- The student then uses those facts to write an accurate answer.

What is RAG?



Why RAG Matters

The Limits of LLMS Alone



Knowledge cutoffs

LLMs trained on static data with fixed cutoff dates provide outdated information and cannot access current events.



Hallucination risk

LLMs without real-data grounding produce confident, plausible-sounding but incorrect answers.



Real consequences

AI errors can have major impacts, as shown when Google Bard's incorrect James Webb Telescope claim contributed to a \$100B stock drop.

RAG reduces these errors by grounding answers in up-to-date, authoritative sources, ensuring reliability and trust.

The fundamental question:

How can we ensure AI responses are accurate, current and grounded in reality?

From Query to Context: The Search Process



When a user submits a question, the RAG system runs a multi-stage process to find and retrieve the most relevant information.



Query embedding

Convert the user's question into a vector representation using the same embedding model used for documents



Similarity search

Compare the query vector against stored document vectors to retrieve the top 3-5 semantically similar chunks



Context packaging

Combine retrieved chunks into a structured context block that will be passed to the LLM for generation



From Query to Context: The Search Process

Example

User asks **"How long does shipping take?"**



System retrieves chunks from shipping SOP and delivery timing policy documents, ensuring the answer reflects current operational procedures.

RAG System Architecture Components

- The system scans knowledge sources like documents, manuals, FAQs, or databases to find relevant information when a user submits a query.
- These retrieved text chunks directly relate to the user's question.

- The system combines the retrieved information with the original user query.
- This process enriches the prompt with factual context, giving the model the precise details needed for a response.

- The AI uses the augmented context to generate a coherent and grounded final answer.
- This ensures the response is fluent, well-structured, accurate and aligned with the source material.

Retrieval (Information Lookup)



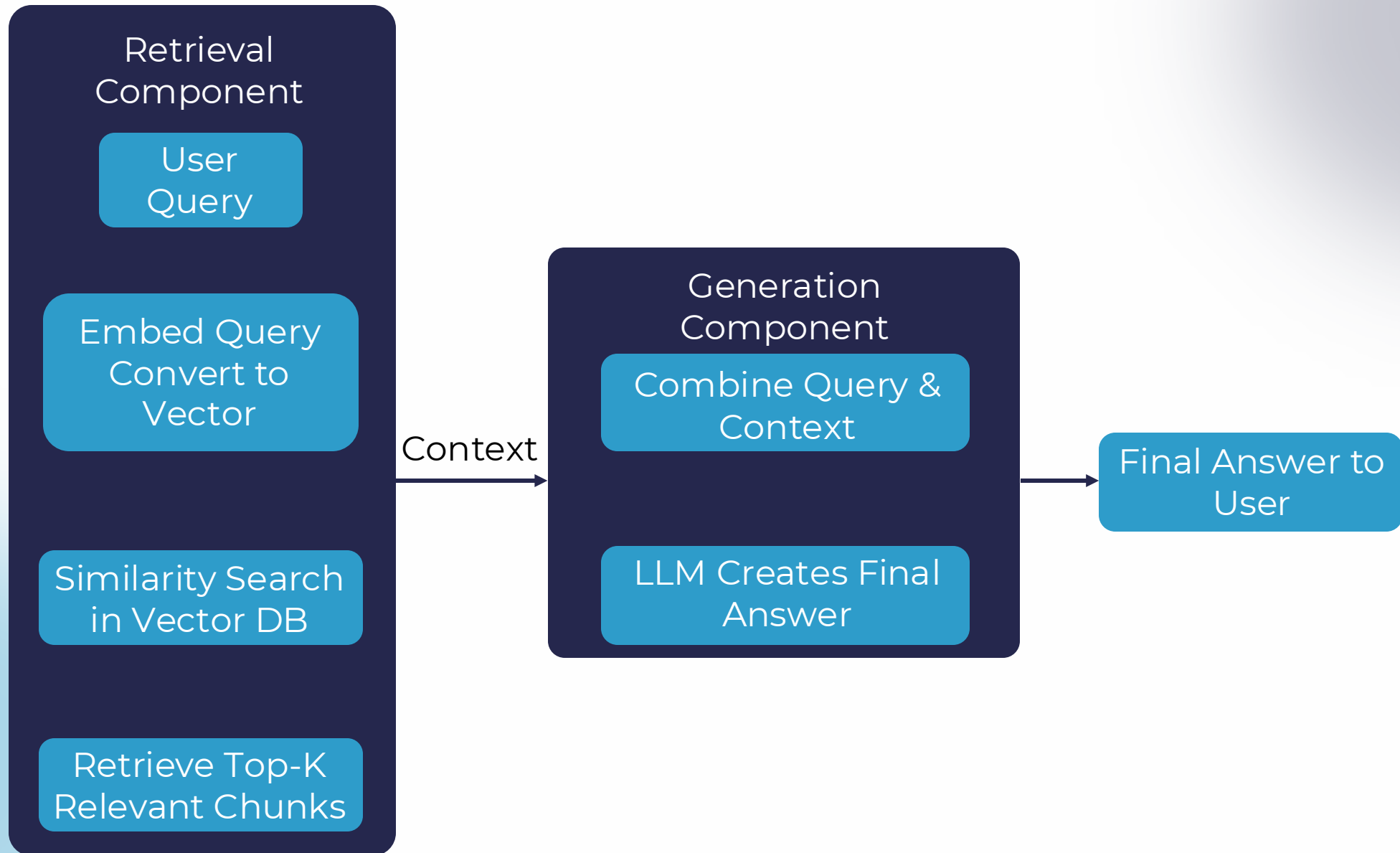
Augmentation (Context enrichment)



Generation (Final response creation)



Understanding Retrieval and Generation Flow



The Retrieval Pipeline: Step-by-Step

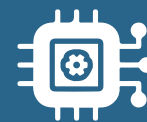
Data collection:

Gather PDFs, websites, manuals and internal documents that contain your knowledge base



Embedding generation:

Convert text chunks into vector representations that capture semantic meaning



Chunking:

Split large documents into manageable pieces (250-1000 characters) for efficient retrieval



Vector storage:

Store embeddings in a vector database optimised for similarity search



The Generation Phase

Creating Accurate Responses

- The LLM receives a structured prompt containing system instructions, the user's question and retrieved context.
- This structure is critical for preventing hallucinations.

SYSTEM:

- You are a grounded, retrieval-augmented assistant.
Answer **only** using the information supplied in the CONTEXT.
If the answer is not fully present in the context, state: **"The provided context does not contain that information."**
Do **not** guess, assume or generate information outside the context.

CONTEXT:

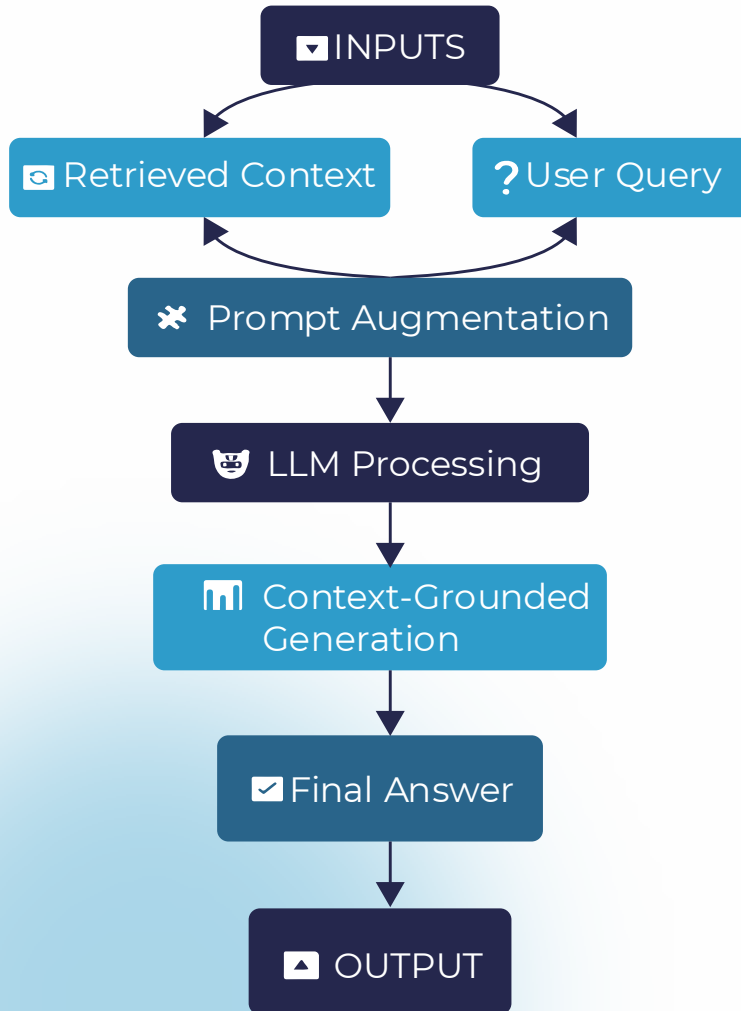
- Standard shipping takes 2–4 days.
- Express delivery takes 1 day.

USER QUESTION:

How long does shipping take?

By limiting the model to the provided context, responses stay grounded in real documentation rather than training data.

Generation Flow in RAG



Inputs:

- Retrieved Context → Relevant documents from vector search
- User Query → Original user question

Process:

- Prompt augmentation → Combines context + query into structured prompt
- LLM processing → Language model processes the augmented prompt
- Context-grounded generation → Creates answer using only provided context

Output:

- Final Answer → Accurate, sourced response

Benefits of Combining Retrieval and Generation



Enhanced accuracy: Provide answers grounded in authoritative, current sources rather than outdated training data, ensuring reliability and precision

Reduced hallucination:

Reference real documents to greatly limit AI fabrications and confabulations



Cost-effective updates:

Update the document database instead of retraining large models when knowledge changes

Transparency & trust:

Enable source citations and provenance tracking, so users can verify information and trust the AI outputs



RAG Step-by-Step Workflow



User query: User submits a question or request to the RAG system



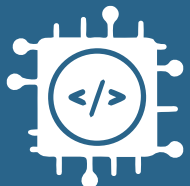
Augmented prompt: LLM receives an enriched prompt combining the original query plus retrieved context



Semantic search: Retrieval model performs semantic search in vector database to find relevant documents



Response generation: LLM generates a grounded, context-aware response based on both its knowledge and retrieved facts

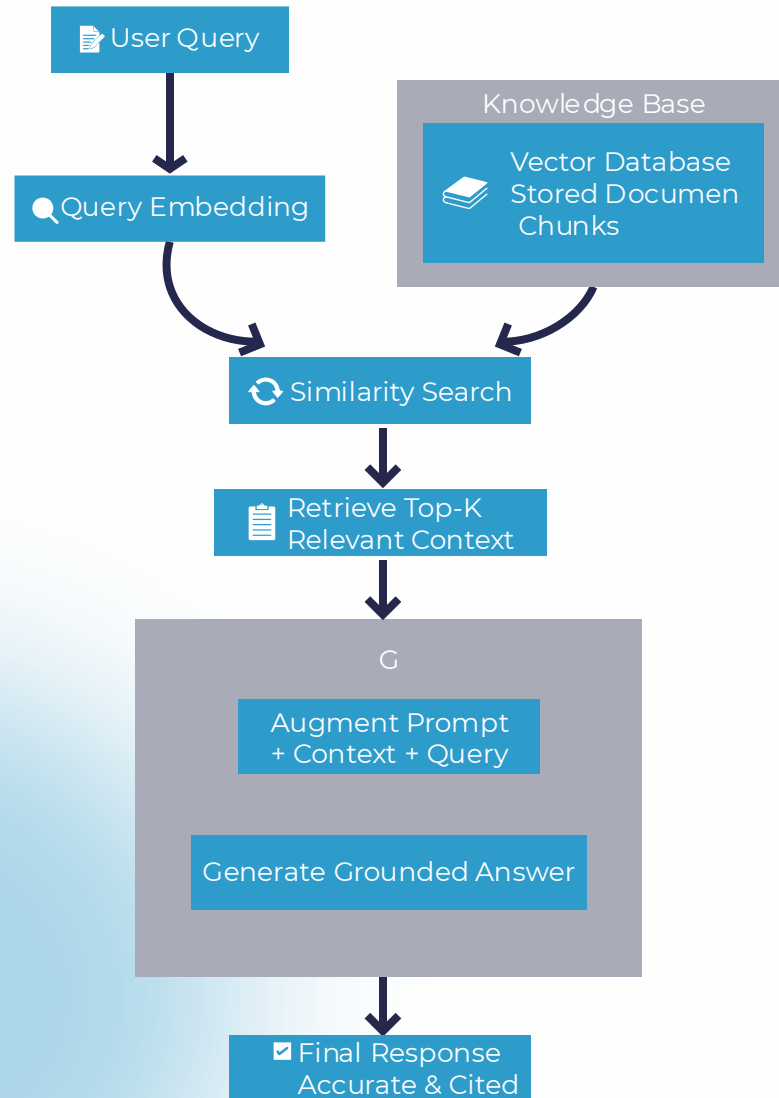


Vector embedding: Retrieved data is chunked and embedded as vectors for efficient search and matching



Output refinement: Post-processing refines and validates output before delivery to ensure quality

RAG Pipeline Flow



- **User query** → Question enters the system
- **Query embedding** → Convert question to numerical vector
- **Similarity search** → Find most relevant documents in vector database
- **Retrieve context** → Get top-K most similar text chunks
- **LLM generation** → Augment prompt with context and generate answer
- **Final response** → Return accurate, sourced answer to user

Example: E-Commerce Support System



Let us walk through an e-commerce customer support example where the system handles returns, shipping and product specification queries.

Collect 20-page return policy PDF, product catalog, warehouse rules

Data ingestion

Chunking & embedding

Create 200 chunks, generate embeddings, store in Pinecone

User asks about refund timeline, system retrieves policy section 4.1

Query processing

Response generation

LLM combines context with query to produce accurate, policy-compliant answer

This architecture scales to thousands of concurrent queries while maintaining accuracy and reducing hallucinations by over 90% compared to standard LLMs.

Key Technologies for RAG

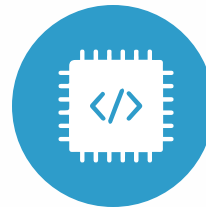
- **Pinecone:** Managed, scalable vector search
- **Weaviate:** Open-source with GraphQL API
- **ChromaDB:** Lightweight, embeddable option
- **FAISS:** Facebook's similarity search library

Vector databases



- **OpenAI:** text-embedding-ada-002
- **Sentence Transformers:** Open-source alternatives
- **Cohere:** Multilingual embeddings

Embedding models



- **LangChain:** Comprehensive RAG toolkit
- **LlamaIndex:** Data framework for LLMs
- **Haystack:** End-to-end NLP framework

RAG frameworks



Benefits of RAG Intergration



Up-to-date answers:

Connects LLMs to live or proprietary data without costly retraining, ensuring responses use the latest information and company-specific knowledge

Cost-effective solution: Avoid expensive model retraining by dynamically augmenting prompts and updating knowledge sources without altering the model architecture



Transparency & trust: Provide responses with citations and source references to boost user confidence and allow accuracy verification

Developer control: Swap or update knowledge sources easily to accommodate new needs, business changes, or domain requirements without system disruption



Use Case: Customer Support



- Consider building a customer support AI for an e-commerce platform like Amazon or Swiggy.
- When a customer asks, "What is the refund policy for late delivery?" the system must provide accurate, up-to-date information.

Use Case: Customer Support



- **Customer query:** Question arrives from the support channel
- **Document retrieval:** System fetches the latest refund policy
- **Contextualised response:** LLM generates an accurate answer using the retrieved policy

This approach ensures responses always reflect current, frequently changing policies.

The AI cannot make up answers because it is constrained to retrieved documentation.

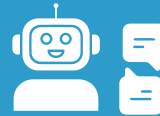
Modern RAG Enhancements

Agentic Retrieval



Parallel query execution

Breaks complex queries into focused subqueries executed simultaneously for faster, more comprehensive results



Structured responses

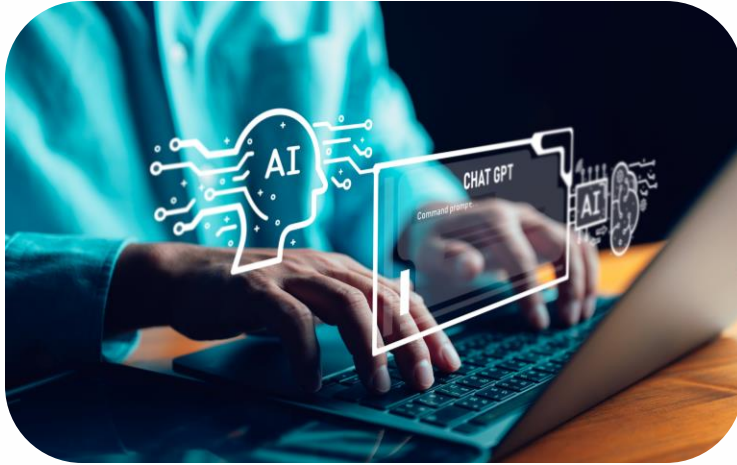
Returns citation-backed answers optimised for chat models with clear source attribution and confidence scores



Semantic ranking

Improves relevance with intelligent ranking algorithms and context-aware query planning that understands user intent

Real World Use Cases



Customer support

Support bots answer company-specific queries by retrieving information from internal manuals, policies, and knowledge bases in real time.



Medical assistance

Healthcare assistants reference current research and clinical guidelines to provide accurate, evidence-based advice.



Financial analysis

Financial analysts access live market data, earnings reports and economic indicators integrated into AI responses for informed decisions.

Success story: Experian's GenAI chatbot used RAG to improve accuracy and prompt handling, reducing errors by 40% while maintaining natural conversation flow.

The Future of Reliable AI

RAG connects static LLM knowledge with dynamic real-world data, creating a powerful hybrid intelligence system

Delivers accurate, trustworthy and context-rich responses that users can rely on for critical decisions

Enables smarter, more adaptable AI solutions that truly understand your data and business needs

Bridging the gap



Empowering accuracy



Building the future



Real-World Applications of RAG



Customer support

Deliver accurate answers instantly from product manuals, FAQs and knowledge bases



Healthcare

Access up-to-date medical research to guide diagnosis and treatment decisions



Enterprise knowledge

Empower employees with precise information from internal documentation



E-commerce

Generate personalised recommendations from product catalogs and customer reviews

Advanced Techniques: Agentic Retrieval

Sophisticated query processing

- Break complex queries into focused, targeted sub-queries
- Execute multiple parallel retrievals for comprehensive coverage
- Synthesise diverse sources into cohesive responses

Enhanced reliability

- Structured responses with explicit citations
- Metadata tracking for source verification
- **Example:** Azure AI Search's agentic pipeline

Modern RAG systems use intelligent agents that adapt retrieval strategies to query complexity and context needs.

Challenges and Considerations



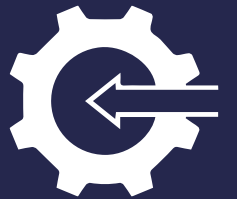
Retrieval quality: Success depends on sophisticated indexing strategies and relevance tuning to find the right documents.

Token constraints: Retrieved context must fit within model input limits, requiring strategic selection and summarisation..



Safety guardrails: Systems need robust controls to avoid bias, prevent hallucinations and ensure ethical outputs.

Integration complexity: Orchestrating retrieval and generation workflows requires careful architectural design and optimization.



The Future: Smarter, More Context-Aware AI

RAG enables AI to stay current without costly retraining

Knowledge updates become simple data refreshes rather than expensive model iterations

Empowers domain-specific, trustworthy AI assistants

Organisations can build specialised systems grounded in their unique knowledge

The key to next-generation intelligent systems

Combining retrieval and generation unlocks AI that truly understands and informs

Let us build AI that's powered by retrieval + generation—intelligent, accurate and ready for the future.

Contextual Embeddings & Relevance Ranking

Improving Retrieval Precision

The Retrieval Challenge in AI Systems



Context loss problem

Traditional Retrieval-Augmented Generation (RAG) struggles with context loss when encoding knowledge base chunks into static vectors



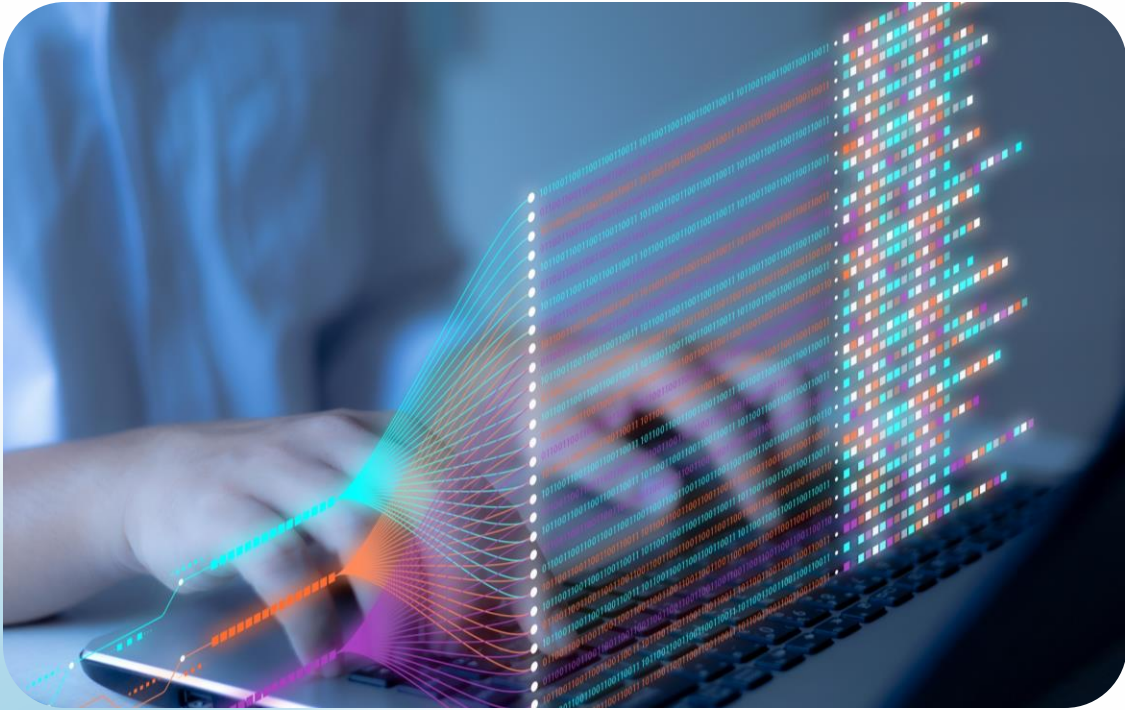
Accuracy impact

AI systems retrieve irrelevant or incomplete information, significantly hurting downstream task accuracy and user trust



Real example: Querying "Error code TS-999" returns general error information instead of the exact technical match needed.

What is Contextual Embedding



- Encode text chunks with surrounding context, dynamically adapting meaning based on the full semantic landscape
- Built with contextual batching, they improve semantic understanding without extensive retraining or high computational cost

Semantic Embeddings with Relevance Ranking

Combined Approach



Semantic embeddings

Capture deep meaning and conceptual relationships but may miss exact keyword matches and unique identifiers



BM25 algorithm

Serve as a lexical matching algorithm that excels at precise term matching, especially for technical identifiers and specific keywords

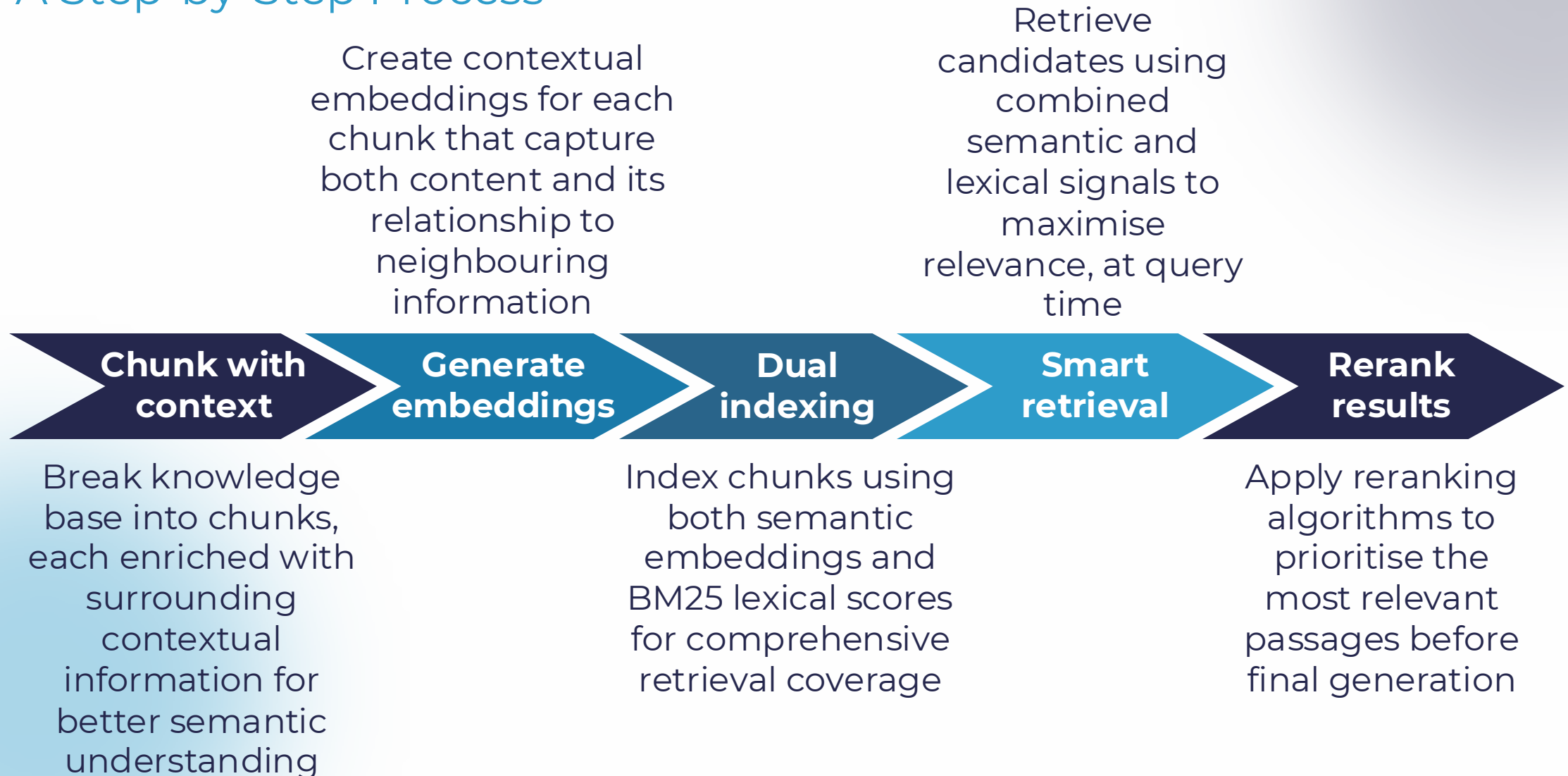


Hybrid power

Combine semantic similarity from embeddings with exact matching from BM25 to achieve higher retrieval precision

How Contextual Retrieval Works

A Step-by-Step Process



Contextual Retrieval: Anthropic's Breakthrough

Anthropic's contextual retrieval introduces two powerful sub-techniques that work in harmony: **Contextual Embeddings** and **Contextual BM25**, fundamentally changing how AI systems access knowledge.

49%

Retrieval failure reduction

Standalone contextual retrieval cuts failed retrievals nearly in half

67%

Retrieval failure reduction

Adding reranking drops failure rates by two-thirds

10x

Understanding boost

Dramatically better comprehension of nuanced queries

Real-World Impact: Improved AI Responses



Customer support

Support bots retrieve exact product information faster, reducing resolution time and improving customer satisfaction.



Legal analysis

Legal AI systems find precise case precedents and statutory references, enhancing analysis quality and lawyer productivity.



Performance gains

Anthropic's Claude API with prompt caching reduces latency by over 2x and cuts costs by up to 90% for retrieval tasks.

Future Directions & Scalability

Massive scale

Contextual embeddings enable scaling to enormous knowledge bases that far exceed prompt window limits

Advanced models

Integration with ranked memory-augmented retrieval models (2025 research) further boosts relevance and accuracy

Continuous evolution

Ongoing improvements in reranking algorithms and local context analysis promise even higher precision rates

Unlocking Precision in AI Retrieval



Transform retrieval

Combine contextual embeddings with relevance ranking to shift retrieval from guesswork to precision engineering



Dramatic results

Reduce failed retrievals by up to 67%, enabling smarter, faster and more reliable AI systems



Take action

Adopt contextual retrieval techniques to elevate your AI's knowledge access and user satisfaction

Customising RAG Models

Tailoring for Q&A, Summarisation & More

Why Customised RAG Models

Customise RAG systems to turn basic setups into powerful, task-specific tools with exceptional performance

Tailor retrieval sources and generation parameters to improve accuracy, relevance, and alignment with specific use cases

Integrate domain-specific or business-critical knowledge that generic models cannot access, creating competitive advantage

Bridging the gap



Empowering accuracy



Proprietary knowledge integration



Customisation Strategies for RAG

Unlock RAG's full potential through strategic customisation at every layer of your pipeline:

RAG enables AI to stay current without costly retraining

Train embeddings on domain-specific data (legal documents, medical literature, technical manuals) to improve retrieval precision and semantic understanding

Optimise retrieval algorithms

Adjust indexing, ranking and search parameters to maximise recall and precision for your datasets

Customise the generator

Fine-tune or prompt-engineer the LLM to match the required tone, style, format and detail level

RAG v. Fine-Tuning: When to Use Which?

RAG excellence	Ideal for dynamic, frequently updated data and broad domain coverage	• Customer support systems • Real-time news analysis • Multi-domain applications
Fine-tuning strength	Best for narrow, static tasks requiring ultra-precise responses	• Specialised summarisation • Domain-specific generation • Consistent style outputs
Hybrid approach	Combines both methods for optimal accuracy and flexibility	• Retrieval + fine-tuned model • Maximum performance • Task-specific excellence

Real-World Example

Summarisation with Custom RAG

Enhanced retrieval

Improve identification and extraction of key sections using embedding models fine-tuned on industry reports

Customised generation

Tailor the LLM to produce concise, jargon-free executive summaries suited to business stakeholders

Scalable processing

Enable efficient summarisation across diverse document types, formats, and complexity levels

Building Your Custom RAG Pipeline

Key Steps

01

Document preparation

Prepare and intelligently chunk your domain-specific documents for optimal retrieval

02

Embedding selection

Fine-tune or select embedding models optimised for semantic search in your domain

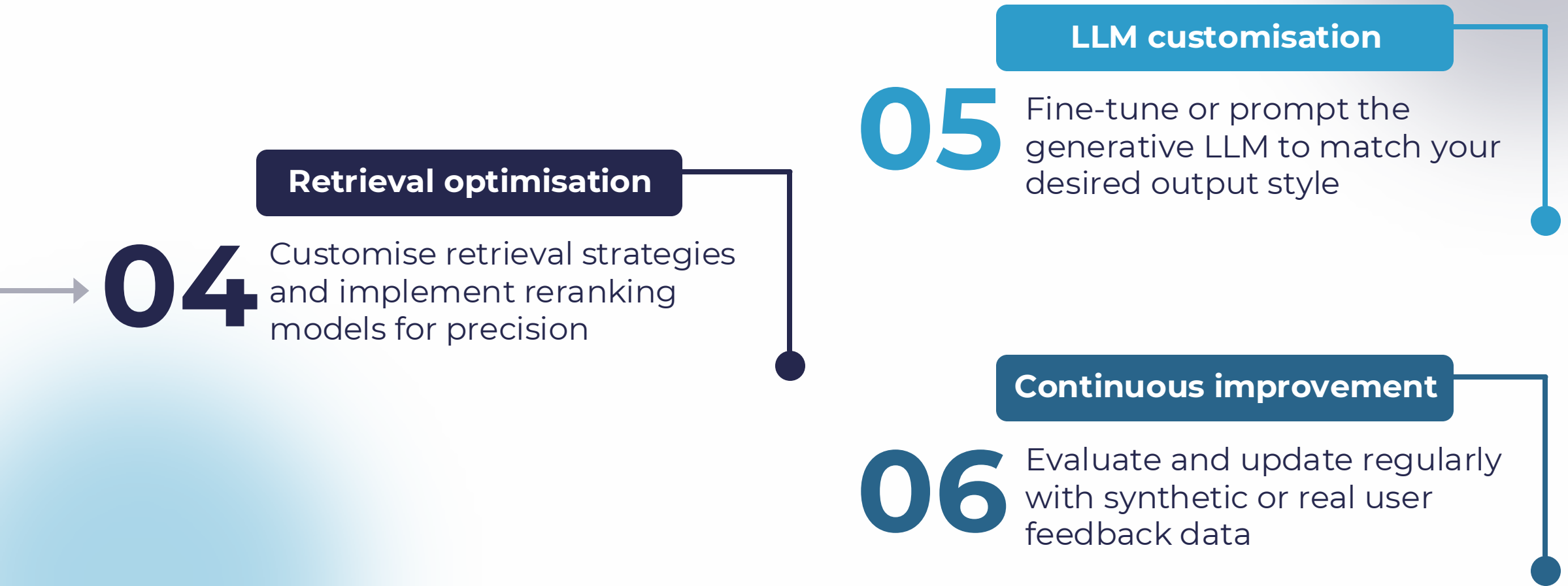
03

Vector indexing

Index documents in a vector database configured for your data characteristics

Building Your Custom RAG Pipeline

Key Steps



Unlocking RAG's Full Potential for Your Tasks

Deliver far more accurate, relevant, and trustworthy AI-generated responses across all use cases

Customise retrieval and generation components to ensure success in Q&A, summarisation, and specialised tasks

Start with basic RAG and progressively enhance it with domain knowledge and task-specific tuning for competitive advantage

Superior outputs



Essential tailoring



Evolutionary approach



Q & A

Thank you